Step-by-Step Guide

Store SmartBank Feedback in Azure Cosmos DB for MongoDB

This PDF explains how to connect an ASP.NET Core MVC project with Azure Cosmos DB using the MongoDB API and verify that feedback data is saved successfully.

Quick Summary

Item	Value
Project	SmartBank ASP.NET Core MVC application
Database Service	Azure Cosmos DB for MongoDB
Database Name	SmartBankDB
Collection	Feedbacks
Main Files	Feedback.cs, MongoService.cs, appsettings.json, Program.cs, Feedback.cshtml

Step 1. Create the Feedback model

Inside the Models folder, create a class named Feedback.cs.

This model stores one feedback record with Id, Name, Email, Comment, and Rating.

The Id field uses MongoDB ObjectId format, so the MongoDB attributes are added above it.

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

public class Feedback
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; }

    public string Name { get; set; }
    public string Email { get; set; }
    public string Comment { get; set; }
    public int Rating { get; set; }
}
```

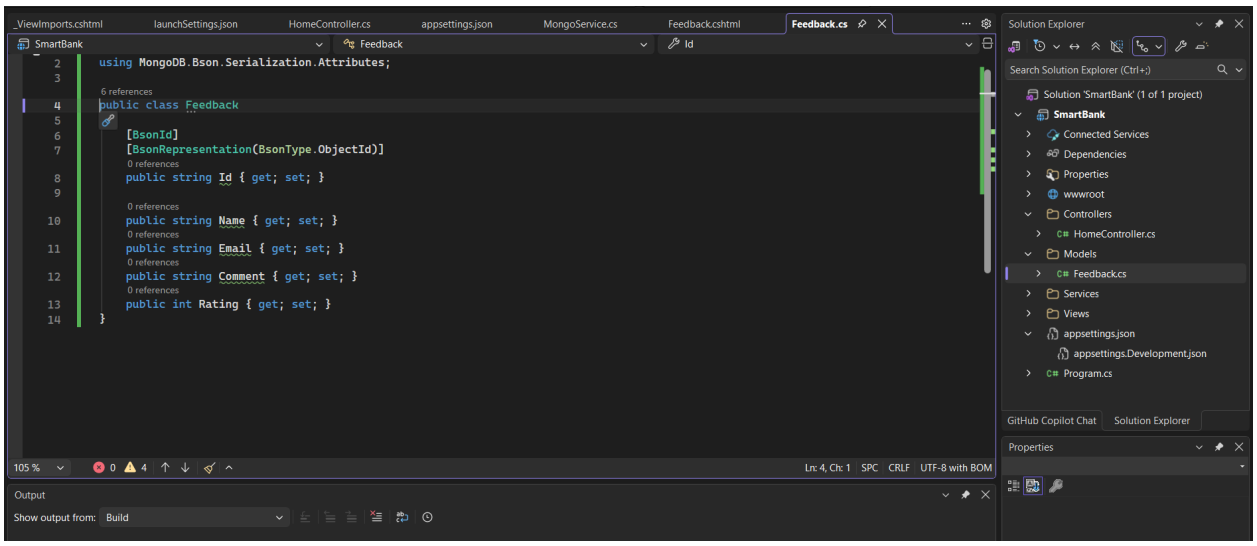

Figure 1. Feedback model in the Models folder.

Step 2. Add MongoDB settings in appsettings.json

Open appsettings.json and create a MongoDbSettings section.

Add the connection string, database name, and collection name.

For safety, keep the real connection string private when you share the project or upload screenshots.

```
"MongoDbSettings": {
  "ConnectionString": "your-mongodb-connection-string",
  "DatabaseName": "SmartBankDB",
  "CollectionName": "Feedbacks"
}
```

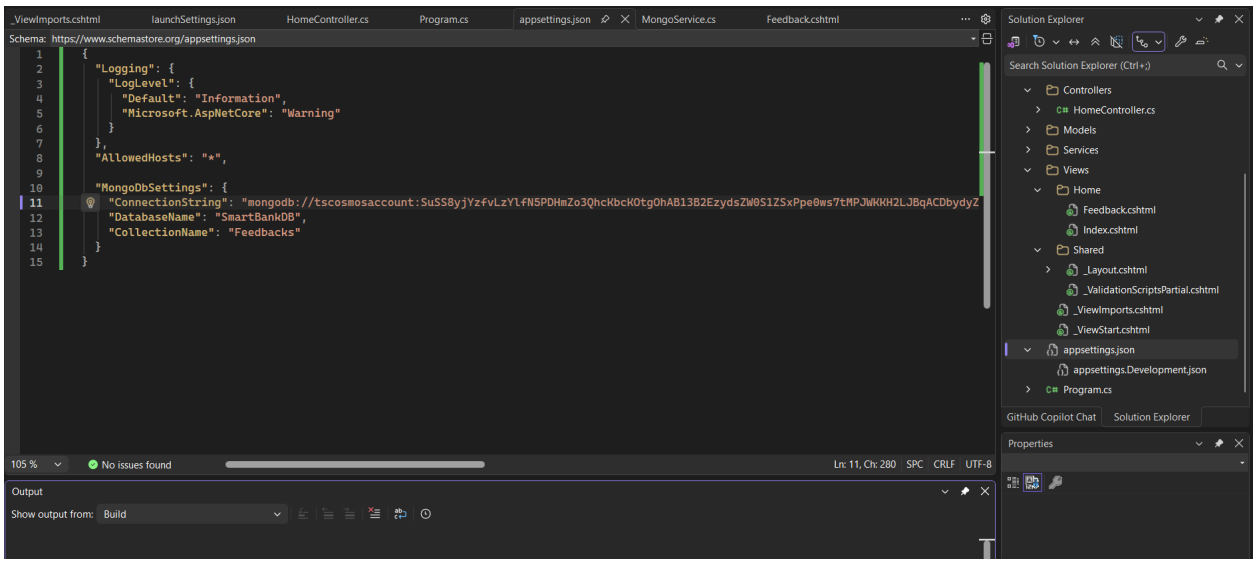

Figure 2. MongoDB configuration added in appsettings.json.

Step 3. Create MongoService to connect with the collection

Inside the Services folder, create a class named MongoService.cs.

This service reads the values from configuration.

It creates the Mongo client, opens the database, connects to the Feedbacks collection, and saves data using InsertOneAsync().

```
using MongoDB.Driver;

public class MongoService
{
   private readonly IMongoCollection<Feedback> _collection;

   public MongoService(IConfiguration config)
   {
      var client = new MongoClient(config["MongoDbSettings:ConnectionString"]);
      var database = client.GetDatabase(config["MongoDbSettings:DatabaseName"]);
      _collection = database.GetCollection<Feedback>(config["MongoDbSettings:CollectionName"]);
   }

   public async Task SaveFeedback(Feedback feedback)
   {
      await _collection.InsertOneAsync(feedback);
   }
}
```

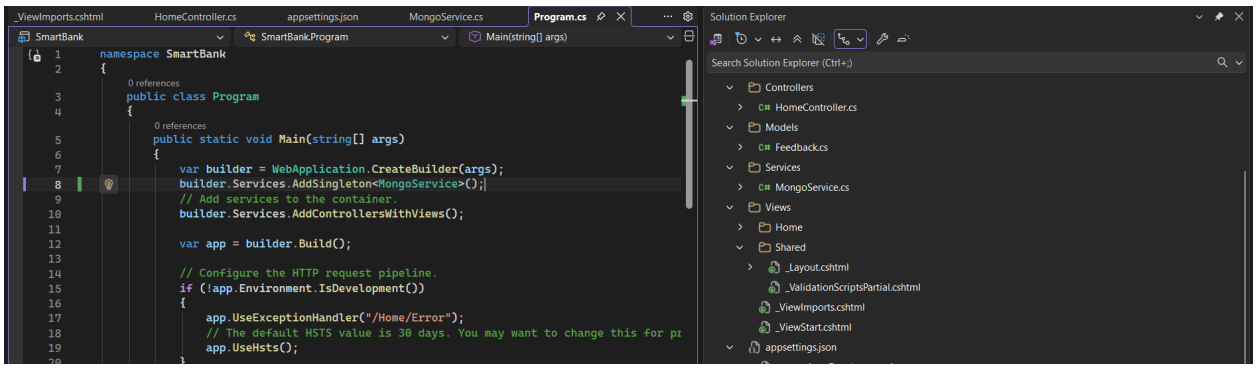

Figure 3. MongoService class used to insert feedback into Cosmos DB.

Step 4. Register MongoService in Program.cs

Open Program.cs and register the service in the dependency injection container.

After this step, the service can be used in controllers through constructor injection.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<MongoService>();
builder.Services.AddControllersWithViews();
```

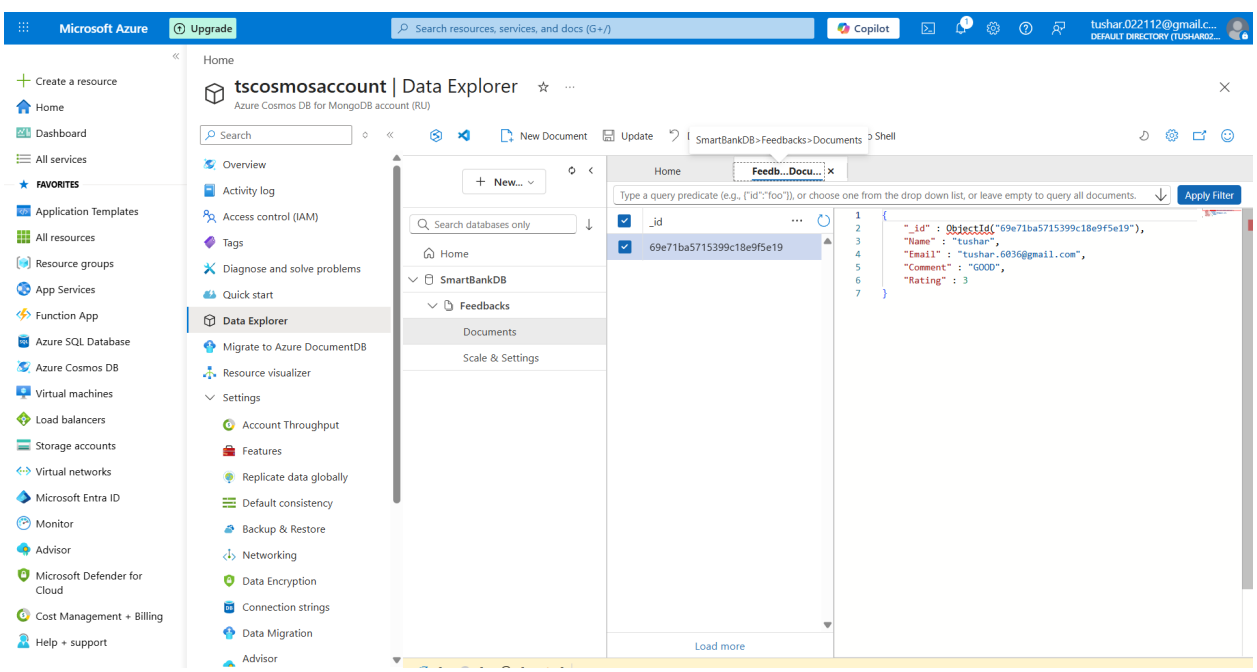

Figure 4. MongoService registered in Program.cs.

Step 5. Create the Feedback page in Views/Home

Add a view named Feedback.cshtml inside Views/Home.

Create a form with input fields for Name, Email, Comment, and Rating.

Set the form method to POST and submit it to the controller action that saves feedback.

```
@model Feedback

<form asp-action="Feedback" method="post">
   <input asp-for="Name" placeholder="Enter name" />
   <input asp-for="Email" placeholder="Enter email" />
   <textarea asp-for="Comment" placeholder="Enter comment"></textarea>
   <input asp-for="Rating" type="number" min="1" max="5" />
   <button type="submit">Submit</button>
</form>
```

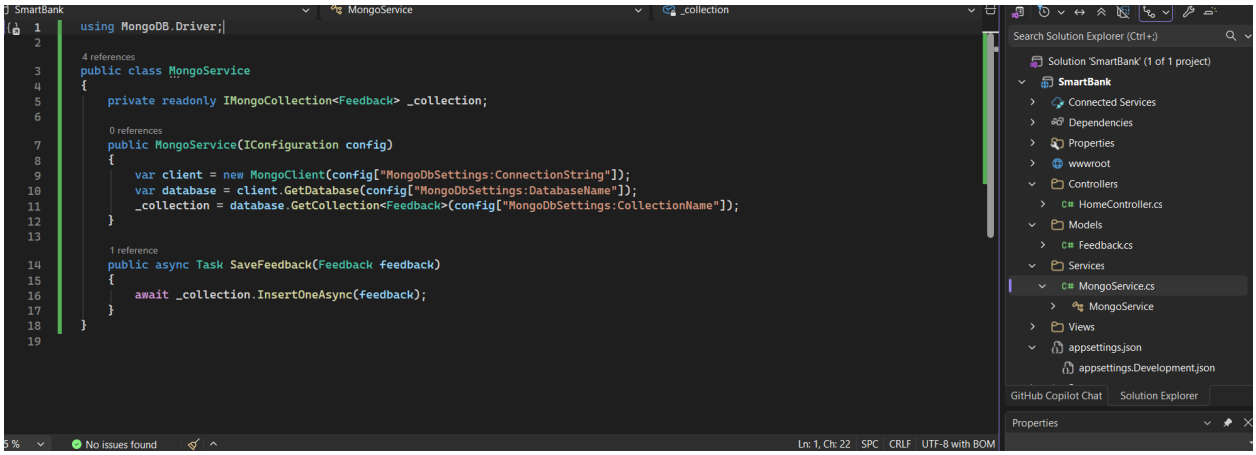

Figure 5. Feedback.cshtml created inside Views/Home.

Step 6. Add controller actions to open and save feedback

Use HomeController or another controller to return the Feedback page on GET request.

On POST request, receive the Feedback model and call MongoService.SaveFeedback(feedback).

After saving, you can show a success message or redirect back to the page.

```
public class HomeController : Controller
{
   private readonly MongoService _mongoService;

   public HomeController(MongoService mongoService)
   {
      _mongoService = mongoService;
```

```
    }

    [HttpGet]
    public IActionResult Feedback()
    {
        return View();
    }

    [HttpPost]
    public async Task<IActionResult> Feedback(Feedback feedback)
    {
        await _mongoService.SaveFeedback(feedback);
        ViewBag.Message = "Feedback saved successfully";
        return View();
    }
}
```

Step 7. Run the project and submit the form

Start the MVC project from Visual Studio.

Open the Feedback page, enter sample values, and submit the form.

If the connection string and service setup are correct, the record will be inserted into Azure Cosmos DB.

Step 8. Verify the inserted document in Azure Portal

Open Azure Portal and go to your Cosmos DB account.

Open Data Explorer, then expand SmartBankDB > Feedbacks > Documents.

You should see the saved document with fields like Name, Email, Comment, and Rating.

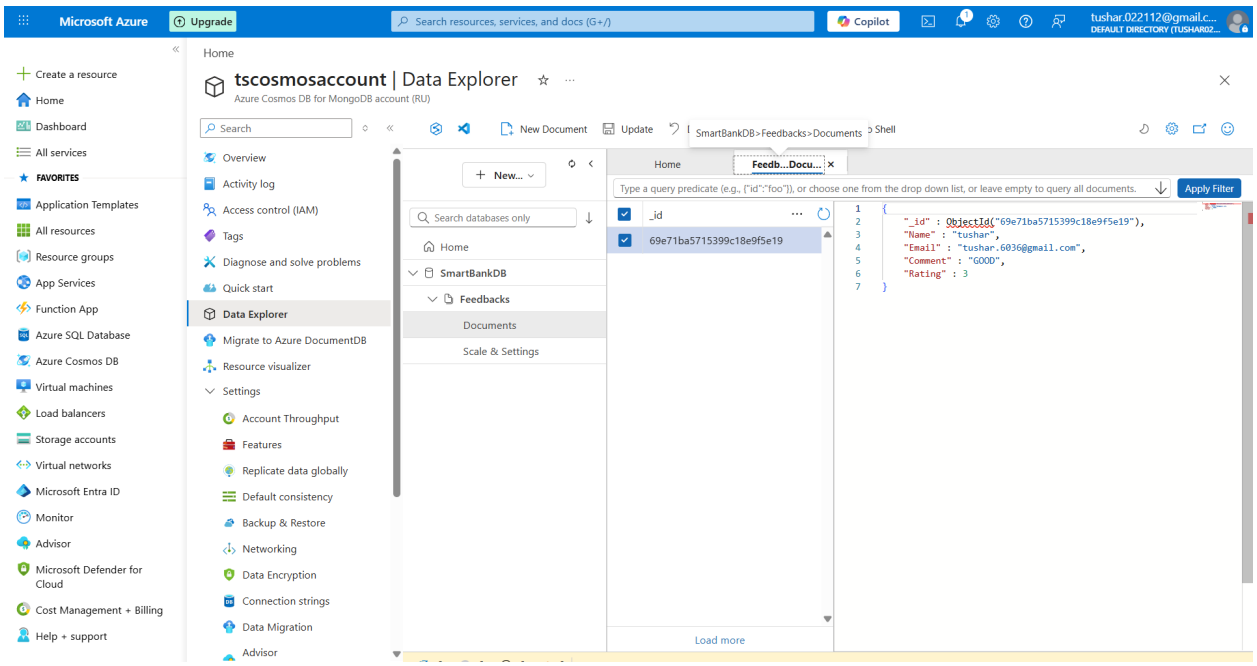

Figure 6. Saved feedback document visible in Azure Cosmos DB Data Explorer.

How the Flow Works

1. User opens Feedback page.
2. User fills the form and clicks Submit.
3. Controller receives the form data.
4. MongoService connects to Cosmos DB MongoDB API.
5. Document is inserted into SmartBankDB > Feedbacks.
6. Data Explorer shows the new saved record.

Important Notes

- Install the MongoDB.Driver NuGet package before building the project.
- Do not expose your real connection string in public screenshots, GitHub, or reports.
- If saving fails, first check the connection string, database name, collection name, and controller POST action.
- Azure Cosmos DB for MongoDB stores data as JSON-like documents, so this model maps naturally to the Feedback class.

Conclusion

By following these steps, your SmartBank MVC project can collect feedback from users and store it in Azure Cosmos DB using the MongoDB API. The screenshots confirm the full flow from code setup to successful document insertion in the Azure portal.